

Mahidol University
Faculty of Information
and Communication Technology

File Operations

ITCS 254 | Lecture 7

Ittipon (Art) Rassameeroj

Agenda



Filesystem



Filesystem Layout



Comparing Files and File Types



Backing Up and Compressing Data

Filesystem

- On many systems (including Linux), the filesystem is structured like a tree.
- The tree is usually portrayed as inverted and starts at what is most often called the root directory, which marks the beginning of the hierarchical filesystem and is also sometimes referred to as the trunk and simply denoted by /.
- Linux supports a number of native filesystem types, expressly created by Linux developers, such as `ext3`, `ext4`, `squashfs`, `btrfs`.
- It also offers implementations of filesystems used on other alien OSs, such as those from Windows (`ntfs`, `vfat`, `exfat`), SGI (`xfs`), IBM (`jfs`), macOS (`hfs`, `hfs+`).
- It is often the case that more than one filesystem type is used on a machine, based on considerations such as the size of files, how often they are modified, what kind of hardware they sit on and what kind of access speed is needed, etc.

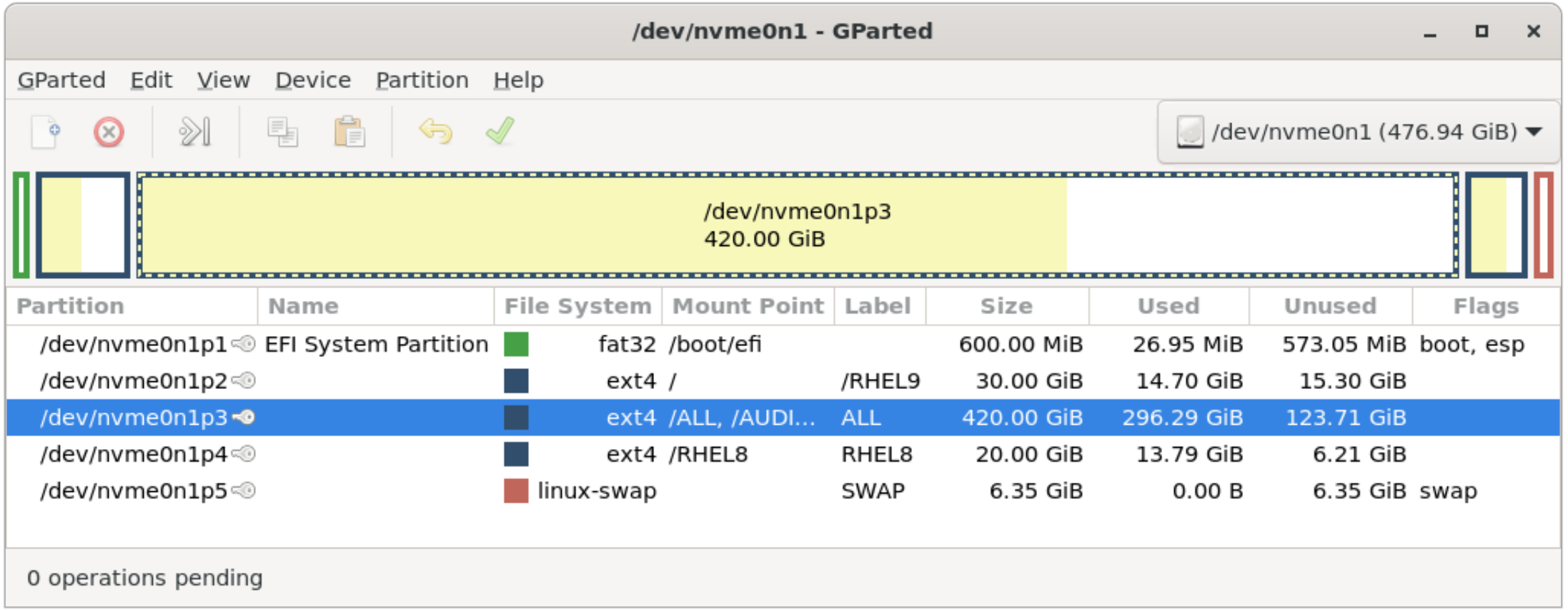
Filesystem

- The most advanced filesystem types in common use are the journaling varieties: `ext4`, `xfs`, `btrfs`, and `jfs`.
- Linux also makes use of network (or distributed) filesystems, where all or part of the filesystem is on external machines.
    - Besides NFS (Network File System) whose usage we will discuss, this includes Ceph, Lustre, and OpenAFS.

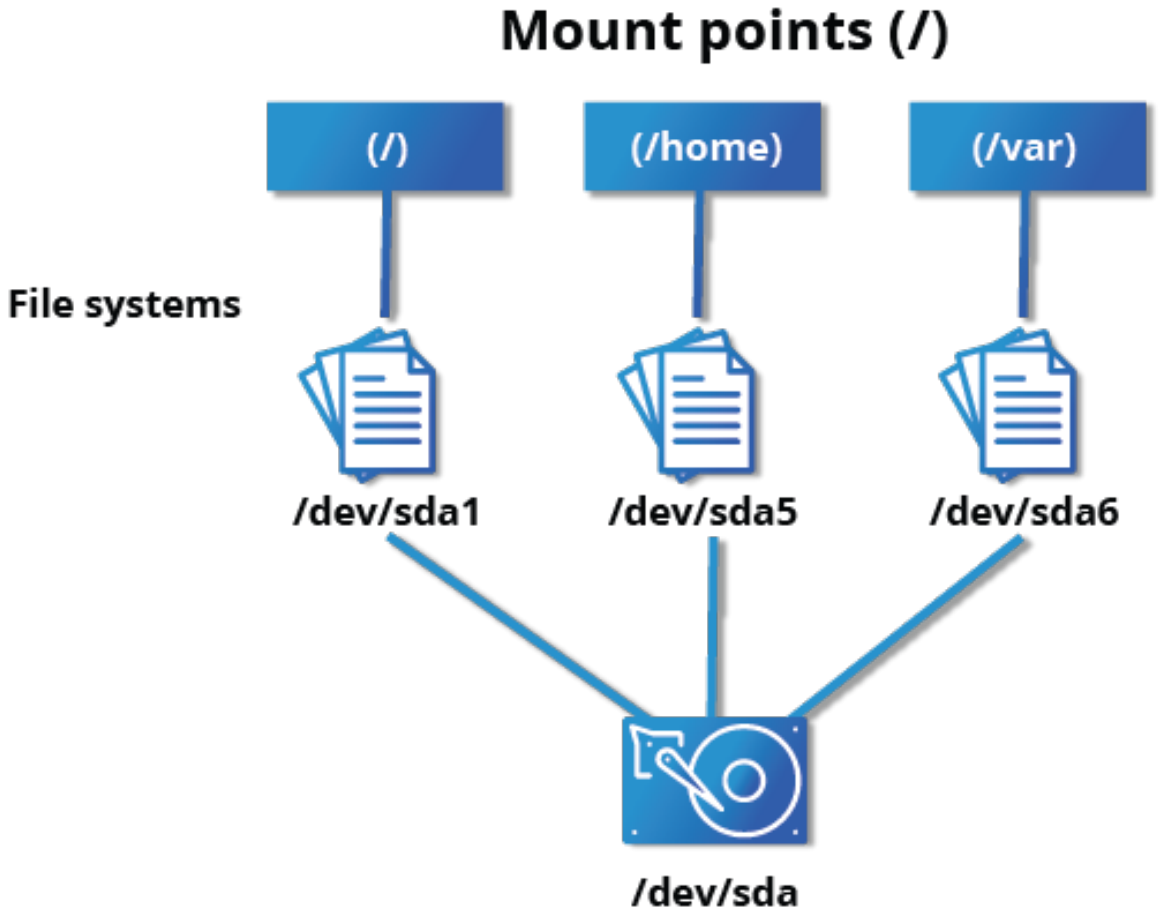
Linux Partitions

- In most situations, each filesystem on a Linux system occupies a disk partition.
- Partitions help to organize the contents of disks according to the kind and use of the data contained.
  - For example, important programs required to run the system are often kept on a separate partition (known as root or `/`) than the one that contains files owned by regular users of that system (`/home`).
- Temporary files created and destroyed during the normal operation of Linux may be located on dedicated partitions.
- One advantage of this kind of isolation by type and variability is that when all available space on a particular partition is exhausted, the system may still operate normally.
- If data is either corrupted through error or hardware failure, or breached through a security problem, it might be possible to confine problems to an area smaller than the entire system.

Linux Partitions

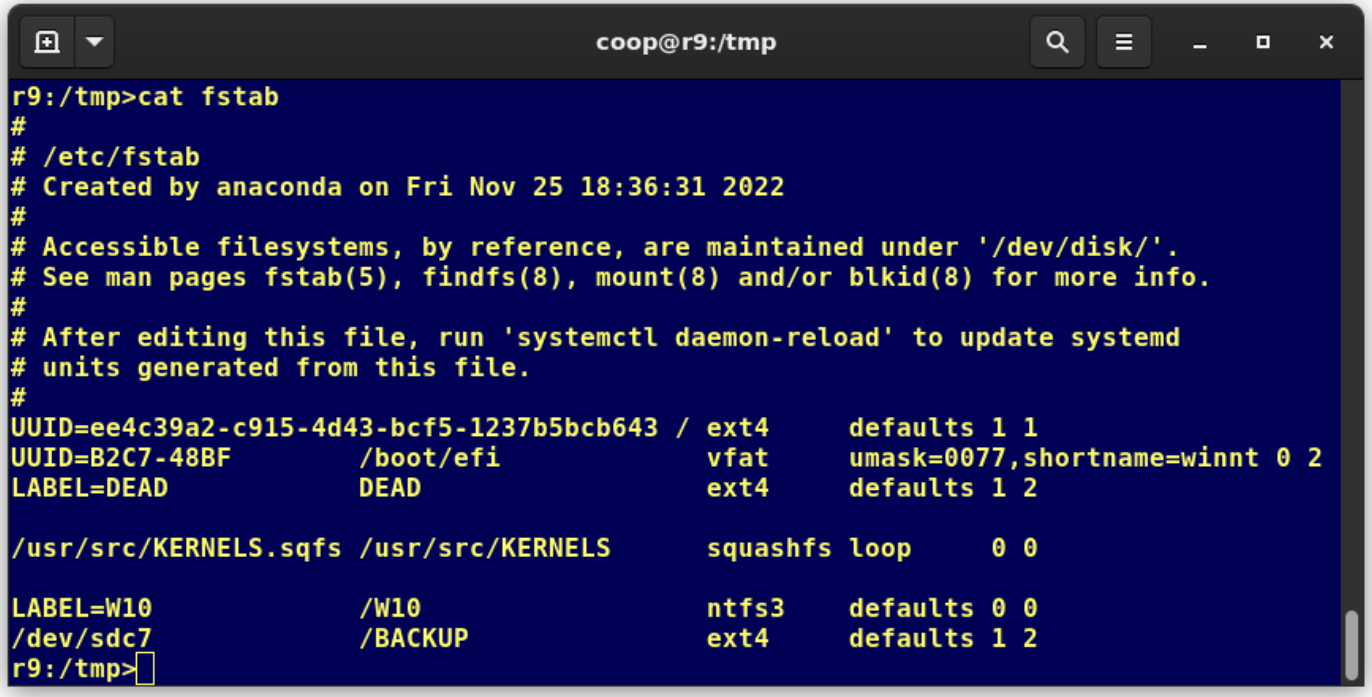


Mount Points

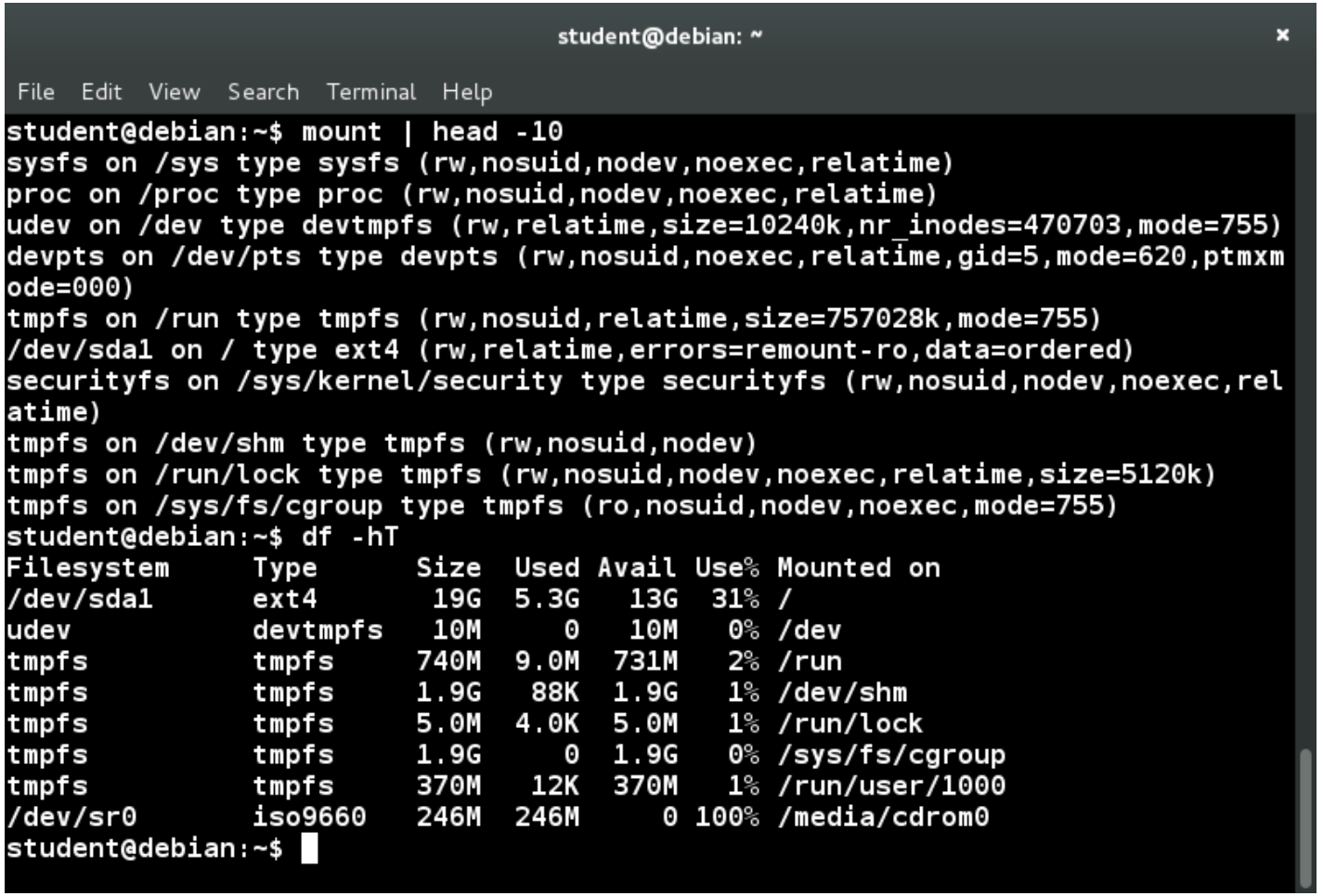


Mounting and Unmounting

- `sudo mount /dev/sda5 /home`
- `sudo umount /home`



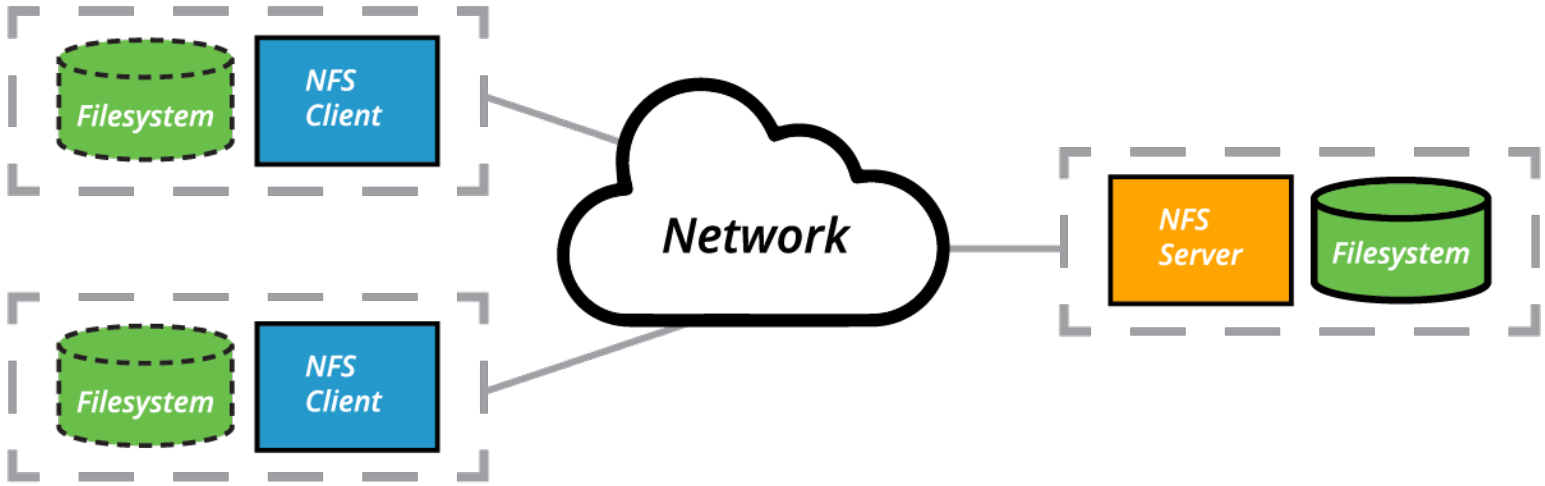
Mounting Information



Mounting Information

```
ubuntu $ cat /proc/mounts | head -20
sysfs /sys sysfs rw,nosuid,nodev,noexec,relatime 0 0
proc /proc proc rw,nosuid,nodev,noexec,relatime 0 0
udev /dev devtmpfs rw,nosuid,noexec,relatime,size=997952k,nr_inodes=249488,mode=755 0 0
devpts /dev/pts devpts rw,nosuid,noexec,relatime,gid=5,mode=620,ptmxmode=000 0 0
tmpfs /run tmpfs rw,nosuid,nodev,noexec,relatime,size=203096k,mode=755 0 0
/dev/vda1 / ext4 rw,relatime 0 0
securityfs /sys/kernel/security securityfs rw,nosuid,nodev,noexec,relatime 0 0
tmpfs /dev/shm tmpfs rw,nosuid,nodev 0 0
tmpfs /run/lock tmpfs rw,nosuid,nodev,noexec,relatime,size=5120k 0 0
tmpfs /sys/fs/cgroup tmpfs ro,nosuid,nodev,noexec,mode=755 0 0
cgroup2 /sys/fs/cgroup/unified cgroup2 rw,nosuid,nodev,noexec,relatime,nsdelegate 0 0
cgroup /sys/fs/cgroup/systemd cgroup rw,nosuid,nodev,noexec,relatime,xattr,name=systemd 0 0
pstore /sys/fs/pstore pstore rw,nosuid,nodev,noexec,relatime 0 0
none /sys/fs/bpf bpf rw,nosuid,nodev,noexec,relatime,mode=700 0 0
cgroup /sys/fs/cgroup/net_cls,net_prio cgroup rw,nosuid,nodev,noexec,relatime,net_cls,net_prio 0 0
cgroup /sys/fs/cgroup/freezer cgroup rw,nosuid,nodev,noexec,relatime,freezer 0 0
cgroup /sys/fs/cgroup/blkio cgroup rw,nosuid,nodev,noexec,relatime,blkio 0 0
cgroup /sys/fs/cgroup/hugetlb cgroup rw,nosuid,nodev,noexec,relatime,hugetlb 0 0
cgroup /sys/fs/cgroup/cpu,cpuacct cgroup rw,nosuid,nodev,noexec,relatime,cpu,cpuacct 0 0
cgroup /sys/fs/cgroup/rdma cgroup rw,nosuid,nodev,noexec,relatime,rdma 0 0
ubuntu $
```

NFS and Network Filesystems



- The most common such filesystem is named simply NFS (the Network Filesystem).
- It has a very long history and was first developed by Sun Microsystems.
- Another common implementation is CIFS (also termed SAMBA), which has Microsoft roots.

Agenda



Filesystem



Filesystem Layout

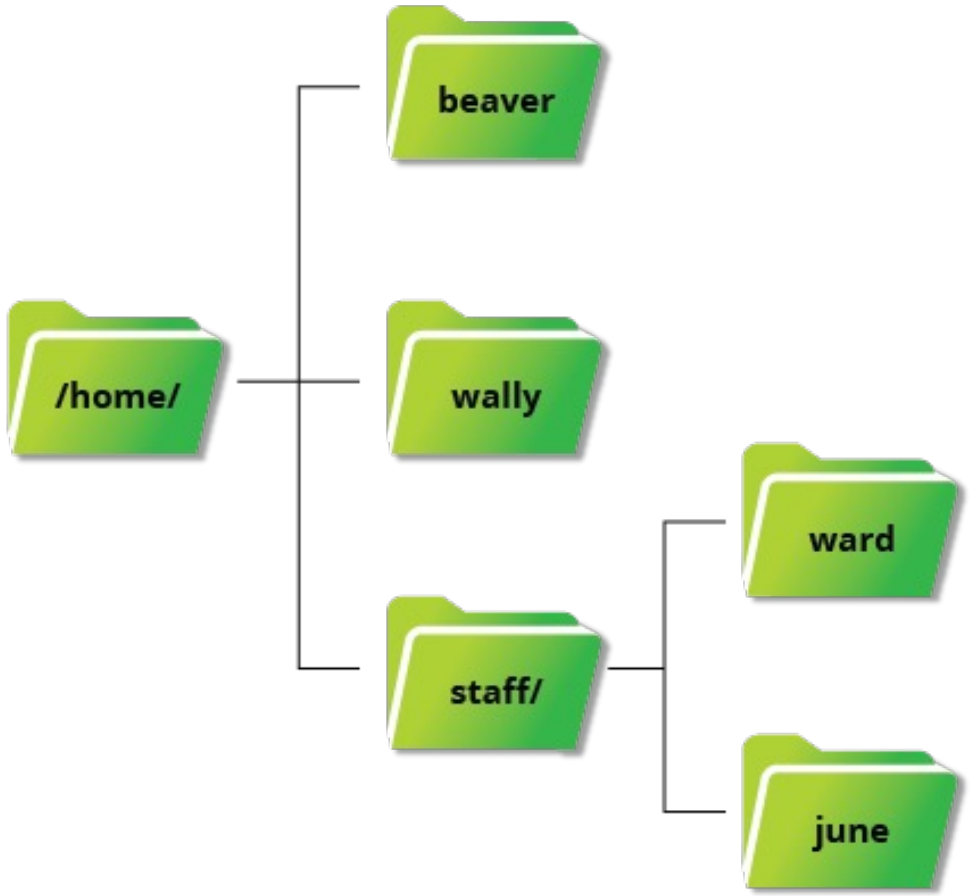


Comparing Files and File Types



Backing Up and Compressing Data

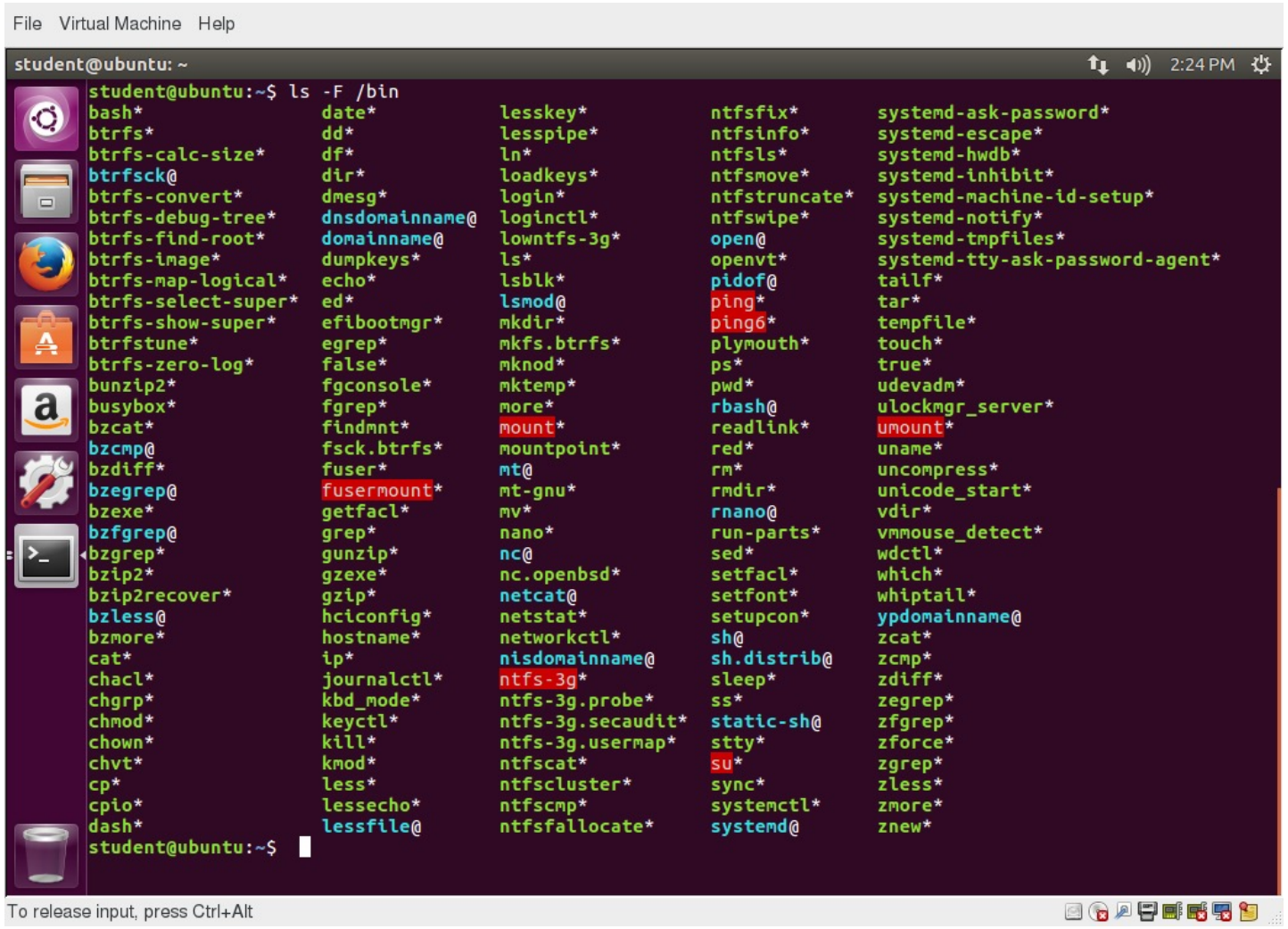
Home Directory



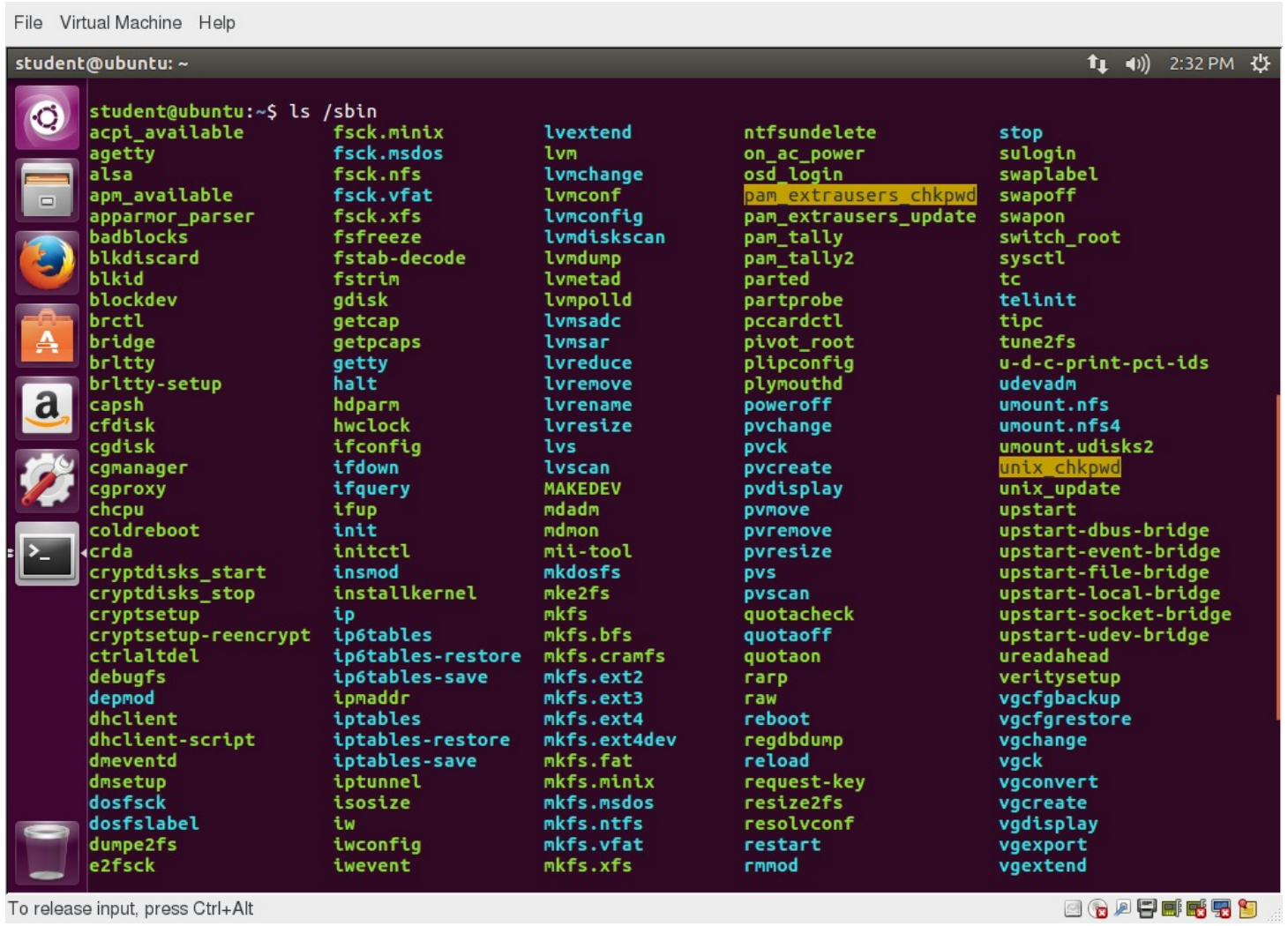
/bin and /sbin Directories

- The /bin directory contains executable binaries, essential commands used to boot the system or in single-user mode, and essential commands required by all system users, such as cat, cp, ls, mv, ps, and rm.
- The /sbin directory is intended for essential binaries related to system administration, such as fsck and ip.
- Commands that are not essential (theoretically) for the system to boot or operate in single-user mode are placed in the /usr/bin and /usr/sbin directories.
- On most Linux distributions today, /usr/bin and /bin are actually just symbolically linked together, as are /usr/sbin and /sbin, so there are really just two directories, not four.

/bin Directory



/sbin Directory



The /proc Filesystem

- Certain filesystems are called pseudo-filesystems because they have no actual permanent presence anywhere on the disk.
- The `/proc` filesystem contains virtual files (files that exist only in memory) that permit viewing constantly changing kernel data.
- `/proc` contains files and directories that mimic kernel structures and configuration information.
- It does not contain real files, but runtime system information, e.g. , system memory, devices mounted, hardware configuration, etc.

- Some important entries in /proc are

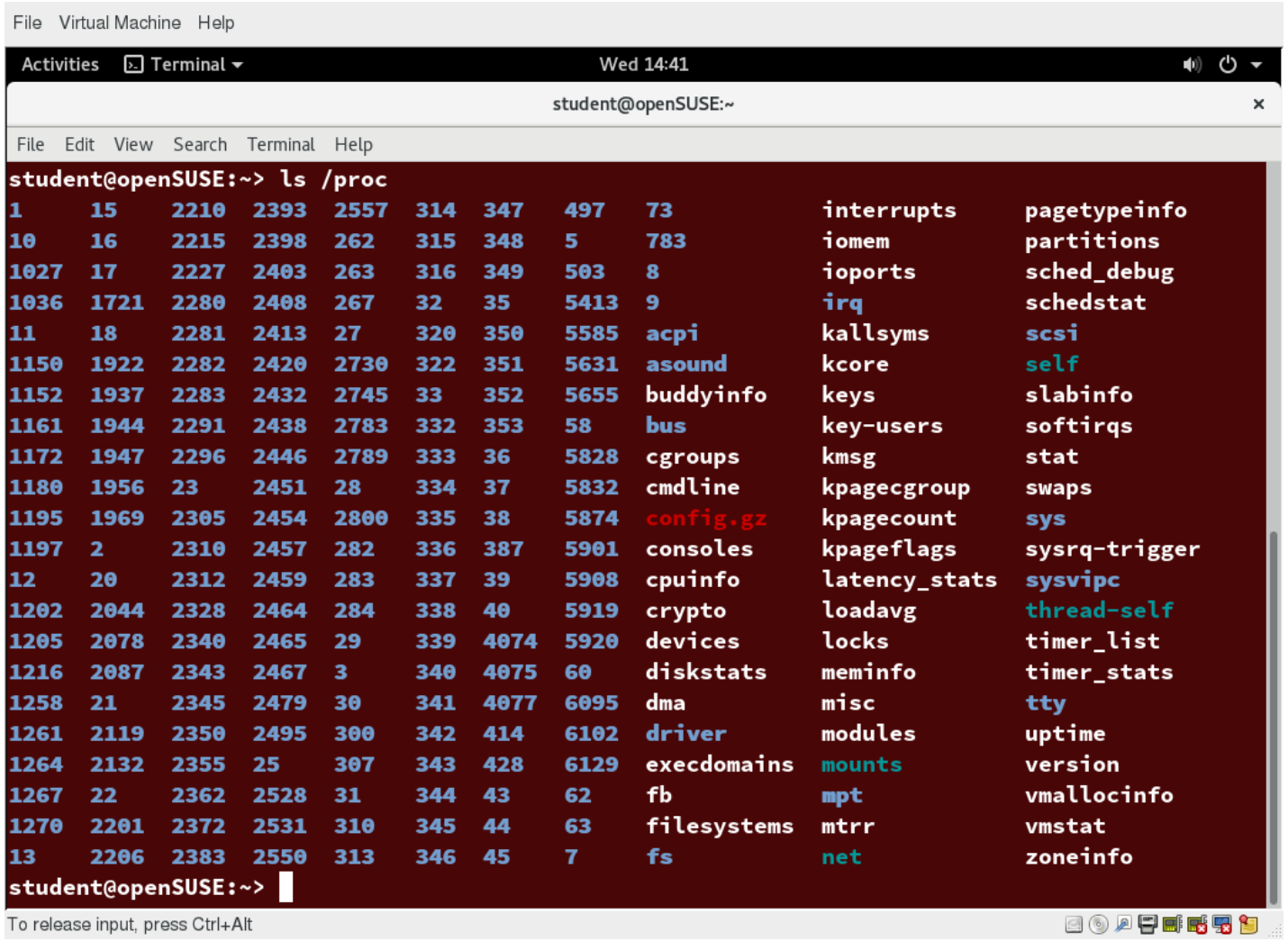
```
/proc/cpuinfo      /proc/interrupts
/proc/meminfo      /proc/mounts
/proc/partitions   /proc/version
```

- /proc has subdirectories as well, including:

```
/proc/<processID>

/proc/sys
```

The /proc Filesystem



The /dev Filesystem

- The `/dev` directory contains device nodes, a type of pseudo-file used by most hardware and software devices, except for network devices.
- This directory is empty on the disk partition when it is not mounted.
- It contains entries which are created by the `udev` system, which creates and manages device nodes on Linux, creating them dynamically when devices are found.
- The `/dev` directory contains items such as:
  - `/dev/sda1` (first partition on the first hard disk)
  - `/dev/lp1` (second printer)
  - `/dev/random` (a source of random numbers).

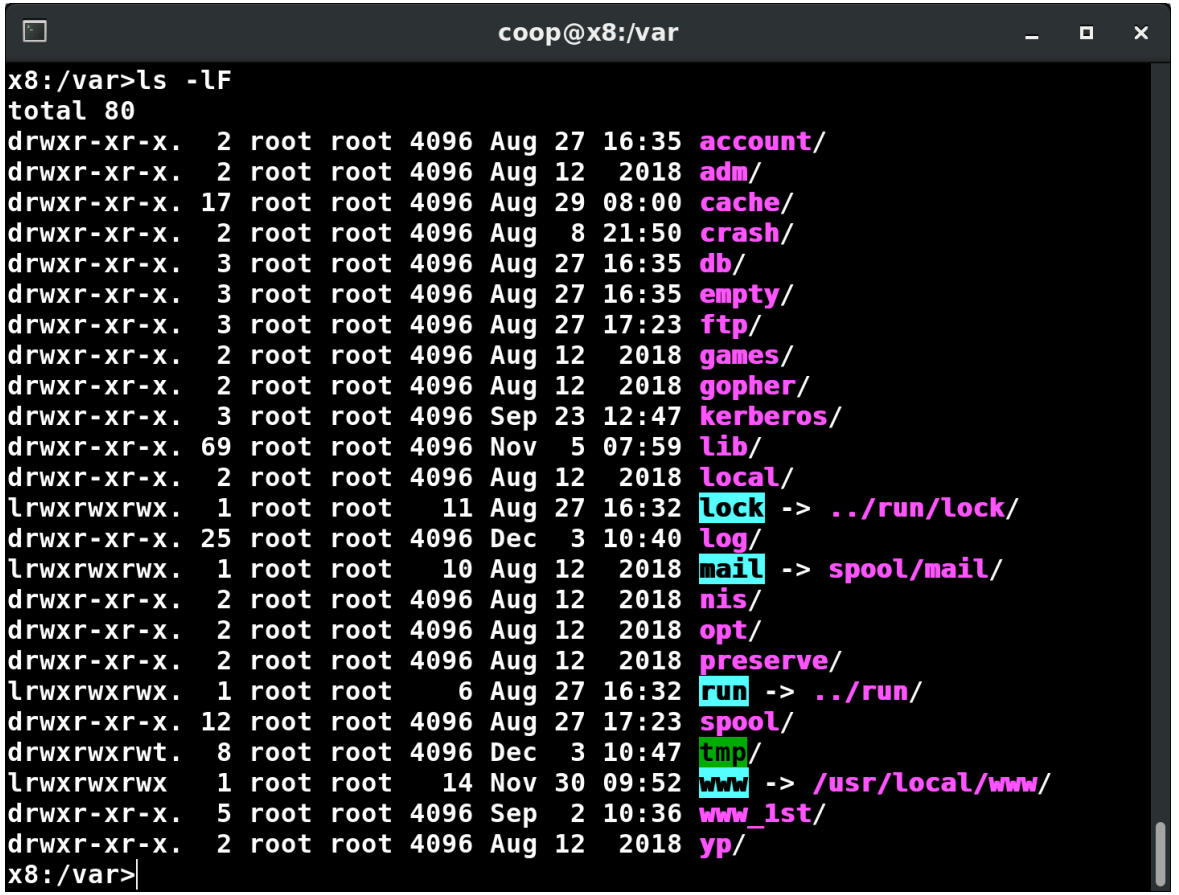
The /dev Filesystem

File Edit View Search Terminal Help

```
c7:/tmp>ls /dev
autofs           dm-5        kmsg              ptmx     sdb7        tty14  tty30  tty47  tty63     vcsa3
block            dm-6        log               ptp0     sequencer   tty15  tty31  tty48  tty7      vcsa4
bsg              dm-7        loop0             pts      sequencer2  tty16  tty32  tty49  tty8      vcsa5
bus              dm-8        loop-control      random   sg0         tty17  tty33  tty5   tty9      vcsa6
char             dm-9        lp0               raw      sg1         tty18  tty34  tty50  urandom   VG
console          dri         lp1               rtc      shm         tty19  tty35  tty51  usb       VG2
core             fb0         lp2               rtc0     snapshot    tty2   tty36  tty52  v4l       vga_arbiter
cpu              fd          lp3               sda      snd         tty20  tty37  tty53  vboxusb   video0
cpu_dma_latency  full        mapper            sda1     stderr      tty21  tty38  tty54  vcs       vmci
disk             fuse        mem               sda2     stdin       tty22  tty39  tty55  vcs1      vmmon
dm-0             hidraw0     memory_bandwidth  sda3     stdout      tty23  tty4   tty56  vcs2      vmnet0
dm-1             hidraw1     mqueue            sda5     tty         tty24  tty40  tty57  vcs3      vmnet1
dm-10            hidraw2     network_latency   sda6     tty0        tty25  tty41  tty58  vcs4      vmnet8
dm-11            hidraw3     network_throughput sdb     tty1        tty26  tty42  tty59  vcs5      vsock
dm-12            hpet        null              sdb1     tty10       tty27  tty43  tty6   vcs6      zero
dm-2             hugepages   nvram             sdb2     tty11       tty28  tty44  tty60  vcsa
dm-3             initctl     pktcdvd           sdb5     tty12       tty29  tty45  tty61  vcsa1
dm-4             input       port              sdb6     tty13       tty3   tty46  tty62  vcsa2
c7:/tmp>
```

The /var Filesystem

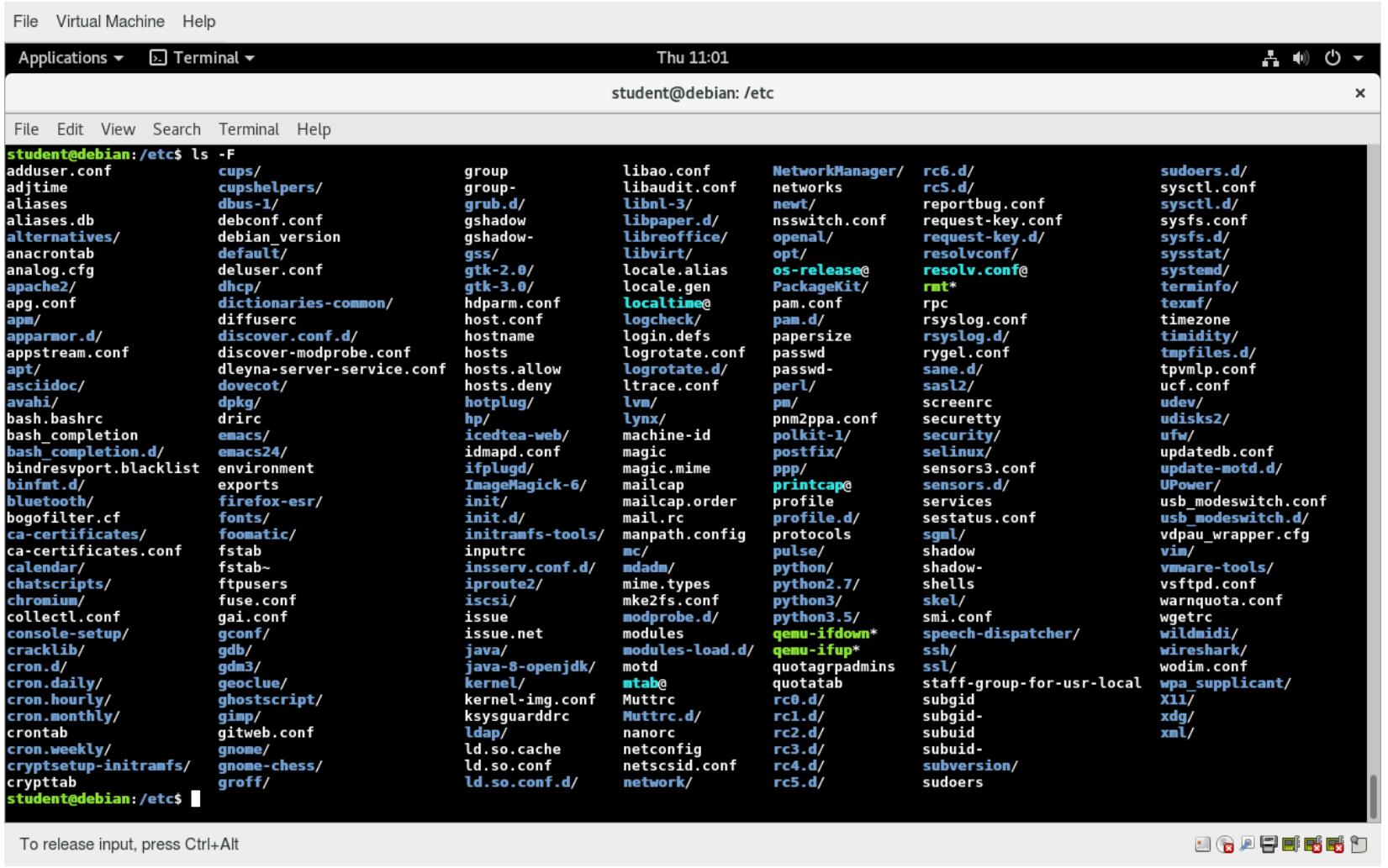
- It contains files that are expected to change in size and content as the system is running (`var` stands for variable)
- System log files: `/var/log`
- Packages and database files: `/var/lib`
- Print queues: `/var/spool`
- Temporary files: `/var/tmp`



The /etc Filesystem

- The /etc directory is the home for system configuration files.
- It contains no binary programs, although there are some executable scripts.
- While some distributions have historically had their own extensive infrastructure under /etc (for example, Red Hat and SUSE have used /etc/sysconfig), with the advent of systemd there is much more uniformity among distributions today.
- It is for system-wide configuration files and only the superuser can modify files there while user-specific configuration files are always found under their home directory.

The /etc Filesystem



The /boot Filesystem

- The `/boot` directory contains the few essential files needed to boot the system.
- For every alternative kernel installed on the system there are four files, each of which these files has a kernel version appended to its name.
  - `vmlinuz`: The compressed Linux kernel, required for booting.
  - `initramfs`: The initial ram filesystem, required for booting, sometimes called `initrd`.
  - `config`: The kernel configuration file, only used for debugging and bookkeeping.
  - `System.map`: Kernel symbol table, only used for debugging.
- The Grand Unified Bootloader (GRUB) files such as `/boot/grub/grub.conf` or `/boot/grub2/grub2.cfg` are also found under the `/boot` directory.

The `/lib` Filesystem

- `/lib` contains libraries (common code shared by applications and needed for them to run) for the essential programs in `/bin` and `/sbin`.
- These library filenames either start with `ld` or `lib`, such as `/lib/libncurses.so.5.9`.
- Most of these are what is known as dynamically loaded libraries (also known as shared libraries or Shared Objects (SO)).
- On some Linux distributions there exists a `/lib64` directory containing 64-bit libraries, while `/lib` contains 32-bit versions.

Removable media: the `/media`, `/run` and `/mnt` Directories

- While historically this was done under the `/media` directory, modern Linux distributions place these mount points under the `/run` directory.
  - For example, a USB pen drive with a label `myusbdrive` for a user named student would be mounted at `/run/media/student/myusbdrive`.
- The `/mnt` directory has been used since the early days of UNIX for temporarily mounting filesystems.
  - These can be those on removable media, but more often might be network filesystems, which are not normally mounted.
  - These can be temporary partitions, or so-called loopback filesystems, which are files that pretend to be partitions.

Additional Directories Under /

Directory Name	Usage
`/opt`	• Optional application software packages
`/sys`	• Virtual pseudo-filesystem giving information about the system and the hardware • Can be used to alter system parameters and for debugging purposes.
`/srv`	• Site-specific data served up by the system • Seldom used.
`/tmp`	• Temporary files; on some distributions erased across a reboot and/or may actually be a ramdisk in memory.
`/usr`	• Multi-user applications, utilities, and data

The /usr directory tree

It contains theoretically non-essential programs and scripts (in the sense that they should not be needed to initially boot the system).

Directory Name	Usage
/usr/include	Header files used to compile applications
/usr/lib	Libraries for programs in /usr/bin and /usr/sbin
/usr/lib64	64-bit libraries for 64-bit programs in /usr/bin and /usr/sbin
/usr/sbin	Non-essential system binaries, such as system daemons and scripts
/usr/share	Shared data used by applications, generally architecture-independent
/usr/src	Source code, usually for the Linux kernel
/usr/local	Data and programs specific to the local machine; subdirectories include bin, sbin, lib, share, include, etc.
/usr/bin	This is the primary directory of executable programs and scripts.

Agenda



Filesystem



Filesystem Layout



Comparing Files and File Types



Backing Up and Compressing Data

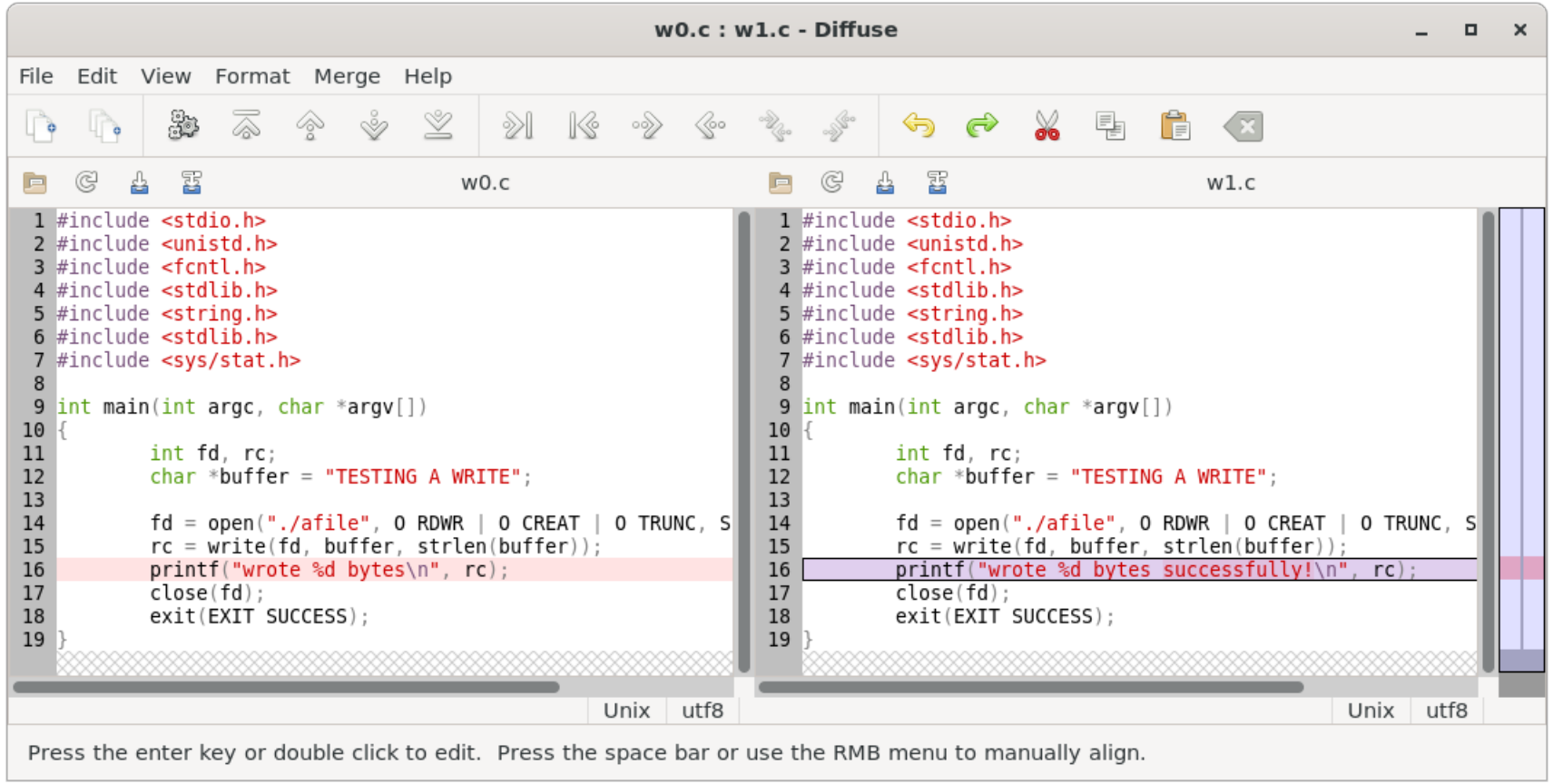
Comparing Files with `diff`

- `diff` is used to compare files and directories.
- `diff [options] <filename1> <filename2>`
- `diff` is meant to be used for text files; for binary files, one can use `cmp`.

Diff Option	Usage
`-c`	Provides a listing of differences that include three lines of context before and after the lines differing in content
`-r`	Used to recursively compare subdirectories, as well as the current directory
`-i`	Ignore the case of letters
`-w`	Ignore differences in spaces and tabs (white space)
`-q`	Be quiet: only report if files are different without listing the differences

Graphical Interface to `diff`: `diffuse`

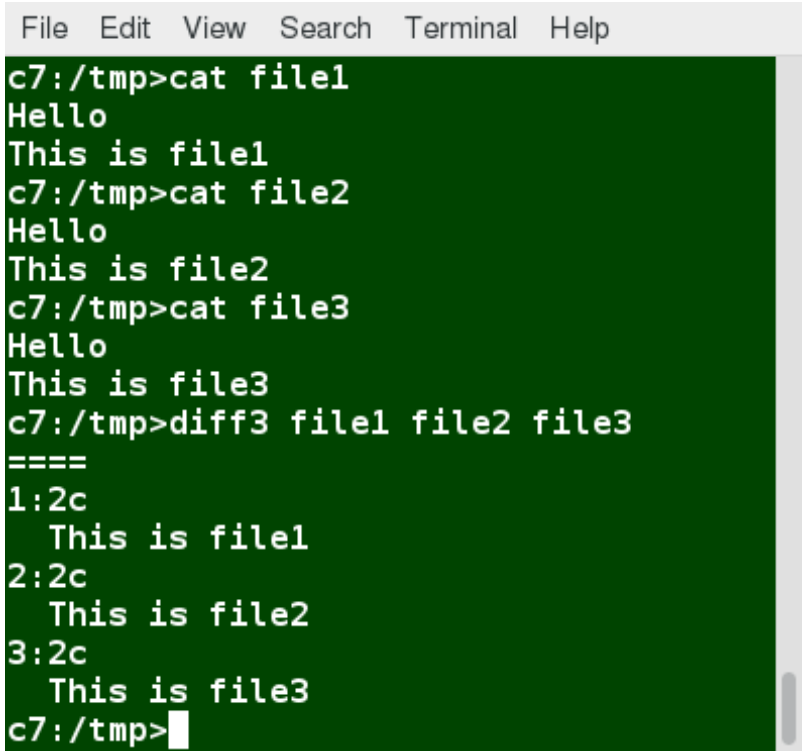
There are multiple graphical interfaces to `diff`, including `diffuse`, `vimdiff`, and `meld`.



Comparing Files with `diff3`

- You can compare three files at once using `diff3`, which uses one file as the reference basis for the other two.

- `diff3 MY-FILE COMMON-FILE YOUR-FILE`



Using patch

- Many modifications to source code and configuration files are distributed utilizing patches, which are applied, not surprisingly, with the patch program.
- A patch file contains the deltas (changes) required to update an older version of a file to the new one.
- Distributing just the patch is more concise and efficient than distributing the entire file.
- The patch files are actually produced by running `diff` with the correct options

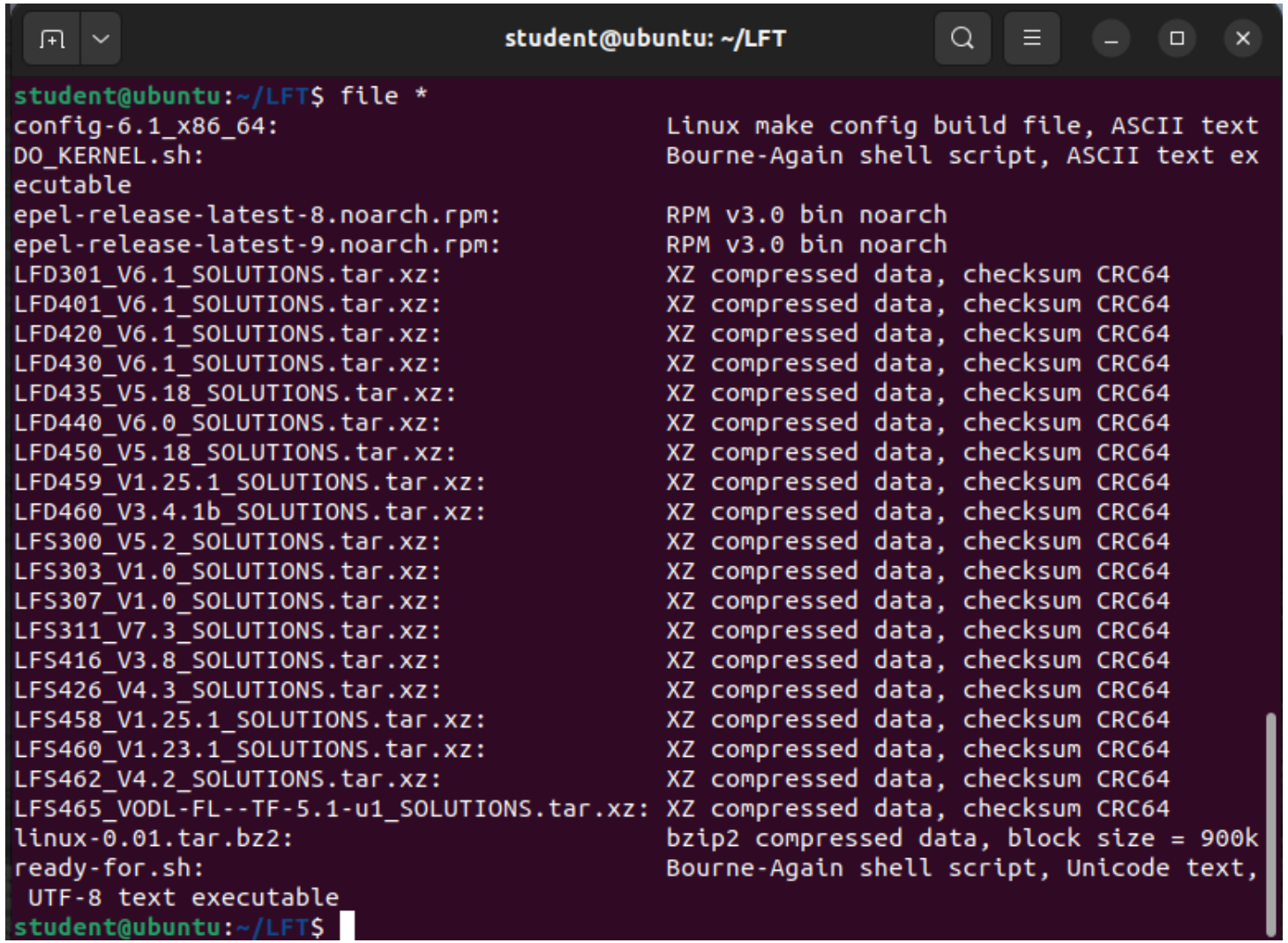
```
$ diff -Nur originalfile newfile > patchfile
```

- To apply a patch, you can just do either of the two methods:

```
$ patch -p1 < patchfile
```

```
$ patch originalfile patchfile
```

Using the `file` Utility



Agenda



Filesystem



Filesystem Layout



Comparing Files and File Types



Backing Up and Compressing Data

Backing Up Data: `cp` and `rsync`

- `cp`
  - It can only copy files to and from destinations on the local machine (unless you are copying to or from a filesystem mounted using NFS)
- `rsync`
  - It checks if the file being copied already exists.
  - If the file exists and there is no change in size or modification time, `rsync` will avoid an unnecessary copy and save time.
  - It can also be used to copy files from one machine to another.
  - `rsync` is very efficient when recursively copying one directory tree to another, because only the differences are transmitted over the network.
  - One often synchronizes the destination directory tree with the origin using the `-r` option

rsync

- `rsync <sourcefile> <destinationfile>`
- `rsync -r project-X archive-machine:archives/project-X`
- `rsync --progress -avrxH --delete sourcedir destdir`
- Note that `rsync` can be very destructive! Accidental misuse can do a lot of harm to data and programs, by inadvertently copying changes to where they are not wanted.
- It is highly recommended that you first test your `rsync` command using the `-dry-run` option to ensure that it provides the results that you want.

Compressing Data

Command	Usage
`gzip`	The most frequently used Linux compression utility
`bzip2`	Produces files significantly smaller than those produced by `gzip`
`xz`	The most space-efficient compression utility used in Linux
`zip`	Is often required to examine and decompress archives from other OSs

- `gzip` has historically been the most widely used Linux compression utility. It compresses well and is very fast.
- `bzip2` has a syntax that is similar to `gzip` but it uses a different compression algorithm and produces significantly smaller files, at the price of taking a longer time to do its work.
    - It is more likely to be used to compress larger files.

Compressing Data

- `xz` is the most space-efficient compression utility frequently used in Linux and is the choice for distributing and storing archives of the Linux kernel.
  - Once again, it trades a slower compression speed for an even higher compression ratio.
  - It is gradually becoming the dominant compression method, especially for large files which may need to be downloaded from the Internet.
- The `zip` program is rarely used to compress files in Linux, it may be needed to examine and decompress archives from other operating systems.
  - It is only used in Linux when you get a zipped file from a Windows user or environment or from Internet downloads.
  - It is a legacy program. It is neither fast nor efficient.

gzip and bzip2 Commands

Command	Usage
`gzip *`	Compresses all files in the current directory; each file is compressed and renamed with a `.gz` extension.
`gzip -r projectX`	Compresses all files in the `projectX` directory, along with all files in all of the directories under `projectX`.
`gunzip foo`	De-compresses `foo` found in the file `foo.gz`. Under the hood, the `gunzip` command is actually the same as `gzip -d`.

Command	Usage
`bzip2 *`	Compresses all of the files in the current directory and replaces each file with a file renamed with a `.bz2` extension.
`bunzip2 *.bz2`	Decompresses all of the files with an extension of `.bz2` in the current directory. Under the hood, `bunzip2` is the same as calling `bzip2 -d`.

xz Command

Command	Usage
`xz *`	Compresses all of the files in the current directory and replaces each file with one with a `.xz` extension.
`xz foo`	Compresses `foo` into `foo.xz` using the default compression level (-6), and removes foo if compression succeeds.
`xz -dk bar.xz`	Decompresses `bar.xz` into bar and does not remove `bar.xz` even if decompression is successful.
`xz -dcf a.txt b.txt.xz > abcd.txt`	Decompresses a mix of compressed and uncompressed files to standard output, using a single command.
`xz -d *.xz`	Decompresses the files compressed using `xz`.

`zip` Command

Command	Usage
`zip backup *`	Compresses all files in the current directory and places them in the `backup.zip`.
`zip -r backup.zip ~`	Archives your login directory (~) and all files and directories under it in `backup.zip`.
`unzip backup.zip`	Extracts all files in `backup.zip` and places them in the current directory.

Archiving and Compressing Data Using tar

- Historically, tar stood for "tape archive" and was used to archive files to a magnetic tape.
- It allows you to create or extract files from an archive file, often called a *tarball*.
- At the same time, you can optionally compress while creating the archive and decompress while extracting its contents.
- You can separate out the archiving and compression stages, but this is slower and wastes space by creating an unneeded intermediary .tar file, as in:

```
$ tar cvf mydir.tar mydir; gzip mydir.tar

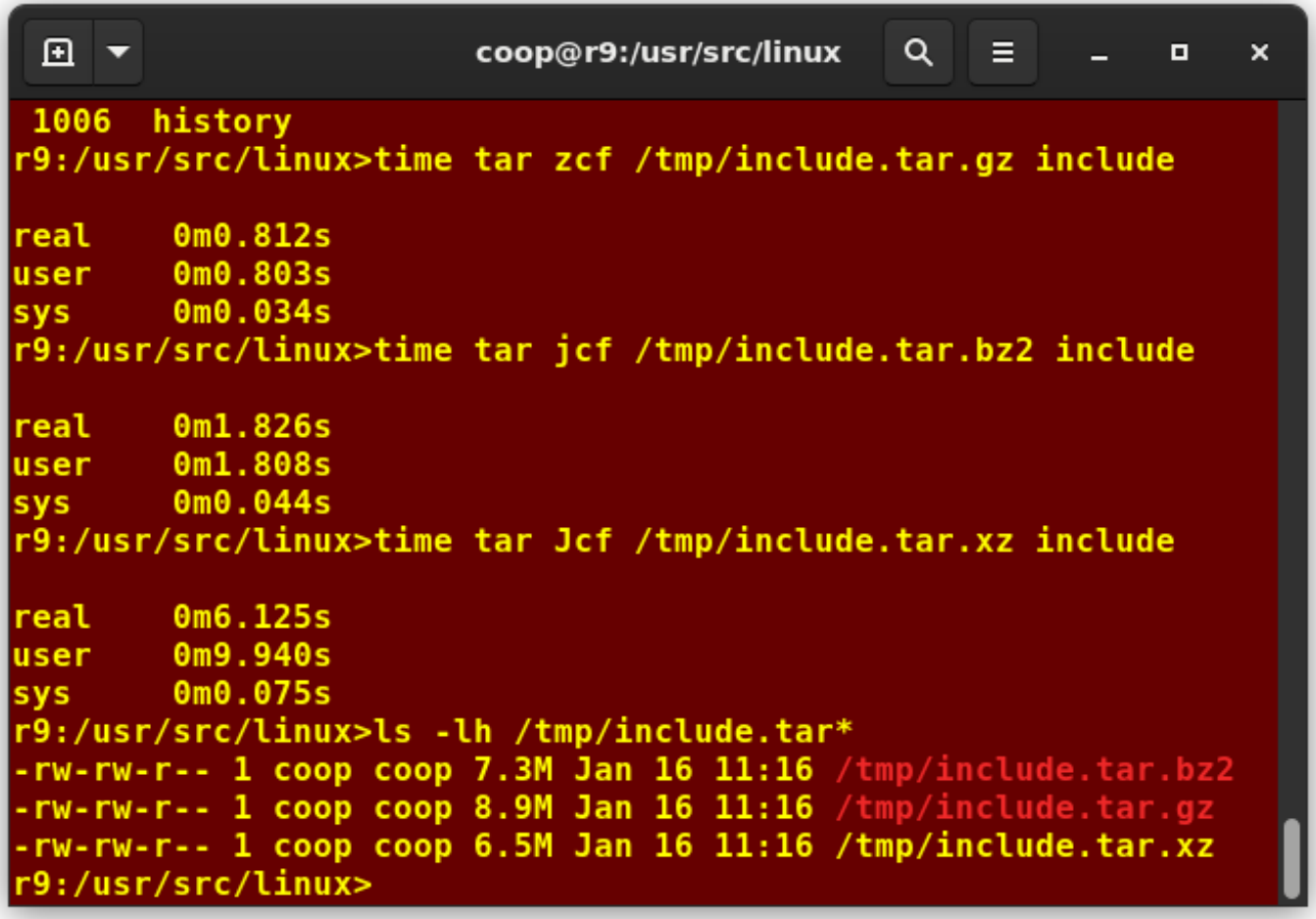
$ gunzip mydir.tar.gz; tar xvf mydir.tar
```



tar Command

Command	Usage
`tar xvf mydir.tar`	Extract all the files in `mydir.tar` into the `mydir` directory.
`tar zcvf mydir.tar.gz mydir`	Create the archive and compress with `gzip`.
`tar jcvf mydir.tar.bz2 mydir`	Create the archive and compress with `bz2`.
`tar Jcvf mydir.tar.xz mydir`	Create the archive and compress with `xz`.
`tar xvf mydir.tar.gz`	Extract all the files in `mydir.tar.gz` into the `mydir` directory. NOTE: You do not have to tell `tar` it is in `gzip` format.

Relative Compression Times and Sizes



Disk-to-Disk Copying (dd)

- The `dd` program is very useful for making copies of raw disk space.
- For example, to back up your Master Boot Record (MBR) (the first 512-byte sector on the disk that contains a table describing the partitions on that disk)

```
dd if=/dev/sda of=sda.mbr bs=512 count=1
```

- `dd if=/dev/sda of=/dev/sdb` to make a copy of one disk onto another, will delete everything that previously existed on the second disk.
  - An exact copy of the first disk device is created on the second disk device.